Sistema Asistivo de Control Facial por Visión Computacional Clásica

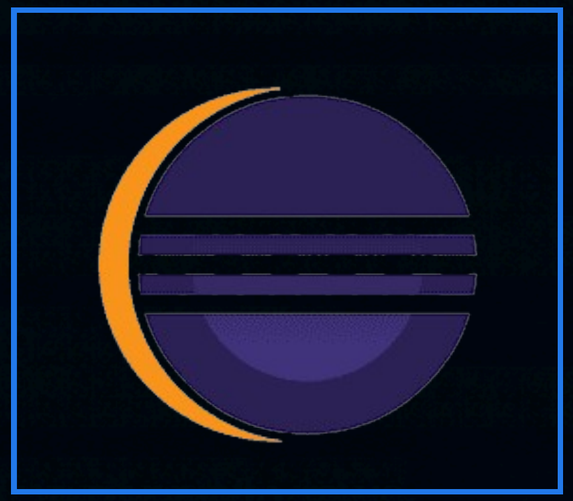
Control del ratón y teclado en **Windows/MAC** mediante **OpenCV**

Python | OpenCV | PyAutoGUI | Tkinter | Winsound



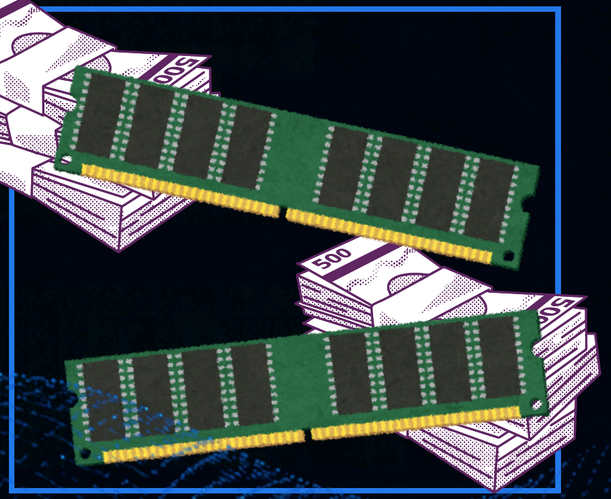
Víctor Serrano Pérez
Justo Cilleros Rodríguez

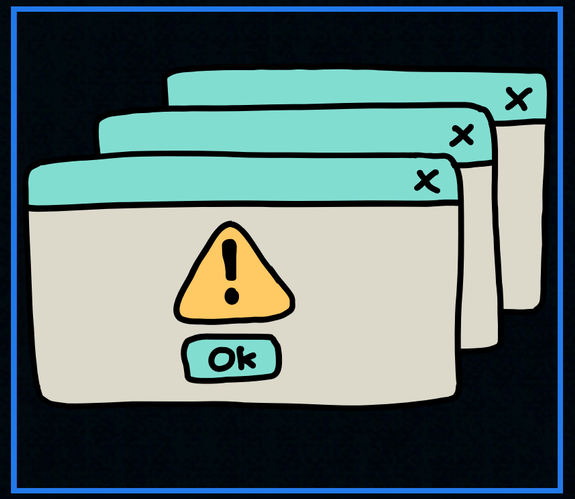
Motivación - Los miedos de un Informático











Propósito del Proyecto y Alcance

Objetivo: accesibilidad digital
de bajo coste con cámara web estándar



CERO LATENCIA

EN TIEMPO REAL

VERSATILIDAD

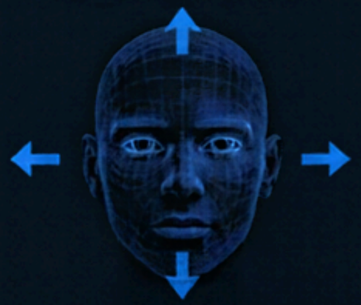
PARA LA GRAN MAYORIA DE SUPUESTOS

ROBUSTEZ

ANTE FALSAS ACTIVACIONES

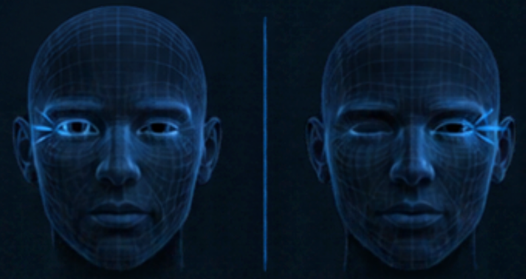
Guía Rápida de Controles y Gestos

Joystick Facial



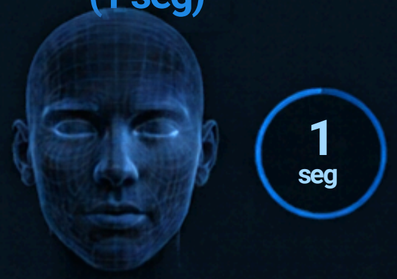
Movimiento de cara
para mover el cursor

Guiño Izquierdo / Derecho



Clic izquierdo con guiño Izquierdo
Clic derecho con guiño derecho

Ojos Cerrados Sostenidos (1 seg)



Activa o desactiva
el modo Drag & Drop

Boca Abierta Sostenida (2 seg)



Pausa o reanuda
la aplicación

Doble Apertura Rápida de Boca



Abre el teclado
en pantalla (osk)

Scroll por Bordes



Mueve el cursor a los límites
para hacer scroll

Sonrisa



Abre el panel flotante
de emojis (Win + .)

Joystick Facial y Movimiento del Cursor



```
# ----------------------------------------
# 2. FACIAL JOYSTICK (MOUSE MOVEMENT)
# ----------------------------------------
face_center_x = x + w // 2
face_center_y = y + h // 2

frame_center_x = w_frame // 2
frame_center_y = h_frame // 2

offset_x = face_center_x - frame_center_x
offset_y = face_center_y - frame_center_y

DEADZONE_X = 25
DEADZONE_Y = 10

cursor_vel_x = 0
cursor_vel_y = 0

if abs(offset_x) > DEADZONE_X:
    cursor_vel_x = int((offset_x - np.sign(offset_x) * DEADZONE_X) * 0.6)

if abs(offset_y) > DEADZONE_Y:
    cursor_vel_y = int((offset_y - np.sign(offset_y) * DEADZONE_Y) * 0.6)

if cursor_vel_x != 0 or cursor_vel_y != 0:
    screen_width, screen_height = pyautogui.size()
    curr_x, curr_y = pyautogui.position()

    SAFE_MARGIN = 10
    target_x = curr_x + cursor_vel_x
    target_y = curr_y + cursor_vel_y

    target_x = max(SAFE_MARGIN, min(screen_width - SAFE_MARGIN, target_x))
    target_y = max(SAFE_MARGIN, min(screen_height - SAFE_MARGIN, target_y))

    pyautogui.moveTo(target_x, target_y, duration=0.0)
```

VARIABLES Y CONSTANTES

`DEADZONE_X = 25`
Zona muerta horizontal: desplazamiento mínimo del rostro antes de mover el cursor.

`DEADZONE_Y = 10`
Zona muerta vertical equivalente.

`SAFE_MARGIN = 10`
Margen de seguridad para que el cursor no salga de la pantalla.

`w_frame / h_frame`
Dimensiones del fotograma de la cámara.

`x, y, w, h`
Caja delimitadora de la cara principal detectada.

Guiños para Clic Izquierdo y Derecho



```
# --- Eye Winks (Left / Right Clicks) ---
if len(eyes) >= 2:
    left_eye_counter = 0
    right_eye_counter = 0
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (0, 255, 0), 2)

elif len(eyes) == 1:
    ex, ey, ew, eh = eyes[0]
    cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (0, 255, 255), 2)

    eye_center_x = ex + ew / 2
    half_face_width = w / 2

    if eye_center_x < half_face_width:
        right_eye_counter += 1
        left_eye_counter = 0
        cv2.putText(frame, f"Right Click ({right_eye_counter})", (50, 50),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 165, 255), 2)

        if right_eye_counter == FRAME_THRESHOLD:
            pyautogui.click(button='right')
            rx, ry = pyautogui.position()
            show_click_effect(rx, ry, click_type="right")
            right_eye_counter = 0
    else:
        left_eye_counter += 1
        right_eye_counter = 0
        cv2.putText(frame, f"Left Click ({left_eye_counter})", (50, 50),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

        if left_eye_counter == FRAME_THRESHOLD:
            pyautogui.click(button='left')
            lx, ly = pyautogui.position()
            show_click_effect(lx, ly, click_type="left")
            left_eye_counter = 0
```

VARIABLES Y CONSTANTES

`left_eye_counter = 0`
Contador de fotogramas con guiño del ojo izquierdo.

`right_eye_counter = 0`
Contador de fotogramas con guiño del ojo derecho.

`FRAME_THRESHOLD = 6`
Fotogramas consecutivos necesarios para confirmar un guiño y ejecutar el clic.

`roi_color`
Región de interés de la mitad superior de la cara, para dibujar los ojos.

`show_click_effect`
Reproduce el sonido y muestra la animación de clic en pantalla.

Ojos Cerrados Sostenidos para Drag & Drop




```
    if len(eyes) == 0:
        # No eyes visible (Sustained closed eyes -> Drag & Drop)
        left_eye_counter = 0
        right_eye_counter = 0

        drag_closed_eyes_counter += 1
        drag_state_text = "Dropping..." if drag_active else "Grabbing..."
        cv2.putText(frame, f"Drag & Drop [{drag_state_text}] ({drag_closed_eyes_counter}/{DRAG_FRAME_THRESHOLD})", (50,
50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 165, 255), 2)

        if drag_closed_eyes_counter >= DRAG_FRAME_THRESHOLD:
            drag_active = not drag_active
            try:
                if drag_active:
                    pyautogui.mouseDown(button='left')
                    winsound.Beep(1000, 150)
                else:
                    pyautogui.mouseUp(button='left')
                    winsound.Beep(600, 150)
            except Exception:
                pass
            drag_closed_eyes_counter = 0

    else:
        drag_closed_eyes_counter = 0
```

VARIABLES Y CONSTANTES

`drag_active = False`
Indica si el modo Drag & Drop está activo.

`drag_closed_eyes_counter = 0`
Contador de fotogramas con los ojos cerrados.

`DRAG_FRAME_THRESHOLD = 35`
Fotogramas consecutivos con ojos cerrados para alternar arrastrar/soltar (~1 segundo).

Boca Abierta Sostenida para Pausar / Reanudar



2 seg

```
# --- Pause / Sleep Mode (Sustained open mouth) ---
if mouth_currently_open:
    sustained_open_mouth_counter += 1
    cv2.putText(frame, f"Loading Pause ({sustained_open_mouth_counter}/{PAUSE_MOUTH_THRESHOLD})", (50, 170),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 165, 255), 2)

    if sustained_open_mouth_counter >= PAUSE_MOUTH_THRESHOLD:
        system_paused = not system_paused
        sustained_open_mouth_counter = 0
        if system_paused and drag_active:
            try:
                pyautogui.mouseUp(button='left')
            except Exception:
                pass
            drag_active = False
        try:
            winsound.Beep(800 if system_paused else 1200, 200)
        except Exception:
            pass
else:
    sustained_open_mouth_counter = 0

# --- Skip all actions if system is paused ---
if system_paused:
    cv2.putText(frame, "--- SYSTEM PAUSED ---", (50, 90),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
    cv2.putText(frame, "Keep mouth open to reactivate", (50, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
    return frame
```

VARIABLES Y CONSTANTES

system_paused = False
Estado de pausa del sistema.

sustained_open_mouth_counter = 0
Contador de fotogramas con la boca abierta.

PAUSE_MOUTH_THRESHOLD = 60
Fotogramas con la boca abierta para alternar la pausa (~2 segundos).

Doble Apertura Rápida de Boca: Teclado Virtual



```
# --- Virtual Keyboard Trigger (Double mouth opening) ---
if mouth_opening_counter > 0:
    frames_since_last_opening += 1
    if frames_since_last_opening > DOUBLE_OPEN_WINDOW_THRESHOLD:
        mouth_opening_counter = 0
        frames_since_last_opening = 0

if mouth_currently_open and not mouth_was_open:
    mouth_opening_counter += 1
    frames_since_last_opening = 0

    cv2.putText(frame, f"Mouth Gesture ({mouth_opening_counter}/2)", (50, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

    if mouth_opening_counter == 2:
        try:
            os_name = platform.system()
            if os_name == 'Windows':
                os.system("start osk")
            elif os_name == 'Darwin':
                os.system("open /System/Library/CoreServices/Applications/Accessibility\ Keyboard.app")
        except Exception:
            pass

        cv2.putText(frame, "--- VIRTUAL KEYBOARD ACTIVATED ---", (50, 170),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

        mouth_opening_counter = 0
        frames_since_last_opening = 0

mouth_was_open = mouth_currently_open
```

VARIABLES Y CONSTANTES

`mouth_opening_counter = 0`
Número de aperturas de boca detectadas en la ventana actual.

`mouth_was_open = False`
Estado de la boca en el fotograma anterior.

`frames_since_last_opening = 0`
Fotogramas transcurridos desde la última apertura.

`DOUBLE_OPEN_WINDOW_THRESHOLD = 45`
Fotogramas máximos entre la primera y la segunda apertura de boca.

Scroll por Bordes



```
#
# 4. EDGE SCROLLING
# ----------------------------------------
screen_width, screen_height = pyautogui.size()
mouse_x, mouse_y = pyautogui.position()
BORDER_MARGIN = 60

if mouse_y < BORDER_MARGIN:
    edge_scroll_counter += 1
    cv2.putText(frame, f"Loading Scroll Up ({edge_scroll_counter}/{EDGE_SCROLL_THRESHOLD})", (50, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    if edge_scroll_counter >= EDGE_SCROLL_THRESHOLD:
        pyautogui.scroll(500)
        edge_scroll_counter = 4
elif mouse_y > screen_height - BORDER_MARGIN:
    edge_scroll_counter += 1
    cv2.putText(frame, f"Loading Scroll Down ({edge_scroll_counter}/{EDGE_SCROLL_THRESHOLD})", (50, 130),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    if edge_scroll_counter >= EDGE_SCROLL_THRESHOLD:
        pyautogui.scroll(-500)
        edge_scroll_counter = 4
else:
    edge_scroll_counter = 0
```

VARIABLES Y CONSTANTES

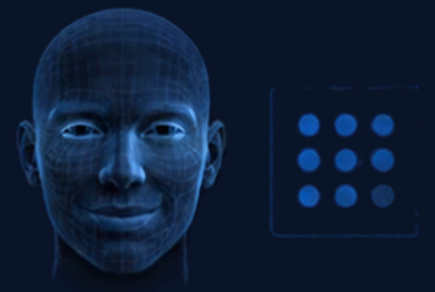
edge_scroll_counter = 0
Contador de fotogramas con el cursor en el borde de la pantalla.

EDGE_SCROLL_THRESHOLD = 8
Fotogramas consecutivos en el borde para disparar el scroll.

BORDER_MARGIN = 60

Franja de borde (en píxeles) que activa el scroll.

Sonrisa Sostenida para el Panel de Emojis



```
# ----------------------------------------
# 5. EMOJI PANEL (SUSTAINED SMILE)
# ----------------------------------------
lower_face_roi = gray[y + int(h*0.5):y + h, x:x + w]
smiles = smile_cascade.detectMultiScale(lower_face_roi, scaleFactor=1.7, minNeighbors=22, minSize=(30, 30))

if len(smiles) > 0:
    if not emoji_locked:
        smile_counter += 1
        cv2.putText(frame, f"Loading Emoji Panel ({smile_counter}/{SMILE_FRAME_THRESHOLD})", (50, 210),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

        if smile_counter >= SMILE_FRAME_THRESHOLD:
            try:
                os_name = platform.system()
                if os_name == 'Windows':
                    pyautogui.hotkey('win', '.')
                elif os_name == 'Darwin':
                    pyautogui.hotkey('ctrl', 'command', 'space')
            except Exception:
                pass
            emoji_locked = True
            smile_counter = 0
else:
    smile_counter = 0
    emoji_locked = False
```

VARIABLES Y CONSTANTES

`smile_counter = 0`
Contador de fotogramas con sonrisa detectada.

`SMILE_FRAME_THRESHOLD = 12`
Fotogramas consecutivos de sonrisa para abrir el panel de emojis.

`emoji_locked = False`
Mecanismo de uso único hasta que la cara se relaje.

`smile_cascade`
Clasificador Haar de sonrisas aplicado a la mitad inferior de la cara.

Uso Técnico de la Librería OpenCV

FUNCIÓN	DESCRIPCIÓN	EJEMPLO VISUAL
detectMultiScale	detección multiescala de rostro, ojos y sonrisa con Haar Cascades	
flip	inversión horizontal para efecto espejo natural	
equalizeHist	normalización de contraste en región ocular	
GaussianBlur	suavizado para eliminar ruido antes del umbralizado	
threshold	binarización estricta para aislar cavidad bucal	
findContours / boundingRect	extracción de contornos y cálculo de apertura vertical	

Conclusiones y Demostración

Sistema fluido y accesible
Respuesta en tiempo real y uso natural

Ligero y eficiente
Bajo consumo y arquitectura optimizada

Listo para Windows y Mac
Operable sin instalaciones complejas

Visión clásica, gran impacto
Control preciso sin IA ni hardware dedicado

¿PREGUNTAS?

